



POLITECNICO DI BARI

**DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELL'INFORMAZIONE
MASTER DEGREE IN AUTOMATION ENGINEERING**

Mobile robotics

Prof. Luca De Cicco

Title:

**Autonomous Navigation System for Vision-Based Human Target Following
in ROS 2 using ultralytics YOLO model**

Student:

Domenico D'Introno

ACADEMIC YEAR 2024-2025



Contents

1	Robot	2
2	Environment	3
3	YOLO Computer Vision Model	4
4	Interfaces	5
5	Custom Nodes	6
5.1	Perceptor	6
5.2	Localizer	8
5.3	Navigator	10
5.4	Initial position publisher	18
6	Launch File	19



1 Robot



2 Environment

3 YOLO Computer Vision Model

4 Interfaces

5 Custom Nodes

This section describes the design and implementation of the three custom ROS 2 nodes: the **Perceptor**, the **Localizer**, and the **Navigator**. Together, they form a pipeline allowing the robot to detect a human in its camera feed, to estimate their 3D position relative to the robot, and to autonomously navigate towards that position using the Nav2 framework.

5.1 Perceptor

The *Perceptor* node is responsible for detecting humans in the camera image stream. It leverages the YOLOv8 neural network model to perform real-time object detection.

Logic pipeline

1. **Subscribes** to the raw RGB image topic `/camera/image_raw`.
2. Applies YOLO to detect the human target.
3. Computes the centroid of the detected bounding box.
4. **Publishes** the centroid as a custom message `CentroidCoords` on `/centroid_coords`.

Code analysis

```
1 import rclpy
2 from rclpy.node import Node
3
4 from sensor_msgs.msg import Image
5 from doom_interfaces.msg import CentroidCoords
6
7 from cv_bridge import CvBridge
8
9 from ultralytics import YOLO
10
11 class Detector(Node):
12
13     def __init__(self):
14         super().__init__('perceptor')
15
16         self.model = YOLO("yolov8m.pt")
17
18         self.bridge = CvBridge()
19
20         self.subscriber_ = self.create_subscription(
21             Image,
22             '/camera/image_raw',
23             self.detect,
24             10
25         )
26
27         self.publisher_ = self.create_publisher(
28             CentroidCoords,
29             '/centroid_coords',
30             10
31         )
32
```

```
33     self.get_logger().info(f"Detection node ready \n")
34
35     def detect(self, msg):
36
37         # Convert a sensor_msgs\Image message in a format that can be used by
38         #   ↳ the YOLO model
39         cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='
40         # Detect a person (class : 0) inside the camera image, use gpu
41         #   ↳ passthrough')
42         results = self.model(cv_image, device="cuda", classes=[0], verbose=
43         # Check that a person has actually been detected, otherwise publish
44         #   ↳ the 'error centroid'
45         if len(results[0].boxes) > 0:
46             [x_min, y_min, x_max, y_max] = results[0].boxes.xyxy[0]
47
48             # Compute centroid (top 25% to avoid targeting the void space
49             #   ↳ between the legs)
50             x_centroid = int((x_min + x_max) / 2)
51             y_centroid = int(y_min + (y_max-y_min) * 0.25)
52
53             centroid = CentroidCoords()
54             centroid.u = x_centroid
55             centroid.v = y_centroid
56             centroid.header = msg.header
57         else:
58             centroid = CentroidCoords()
59             centroid.u = -1
60             centroid.v = -1
61             centroid.header = msg.header
62
63         self.publisher_.publish(centroid)
64
65
66 def main(args=None):
67     rclpy.init(args=args)
68     node = Detector()
69     try:
70         rclpy.spin(node)
71     except KeyboardInterrupt:
72         pass
73     finally:
74         node.destroy_node()
75         rclpy.shutdown()
76
77
78 if __name__ == '__main__':
79     main()
```

The class **constructor** initializes the **perceptor** node: it loads the YOLOv8 model, initializes the CV bridge, which converts between ROS2 Image messages and OpenCV images, subscribes to camera topic, and sets up a publisher for the detected centroid coordinates. The **detect** callback function

runs person detection on the received camera image using YOLO, extracts the bounding box centroid, and publishes it as `CentroidCoords`.

Note that the `model` function takes as arguments `device="cuda"` to specify that the GPU will be used to run the model, plus, the `classes=[0]` argument narrows the model research to just persons.

An important design choice is to compute the person pixel centroid as the **top 25% of its silhouette**, fixing a bug where inconsistent depth measurements and mislocalization occurred when the centroid fell between the legs.

Also, if no person is detected, the node publishes an **'error centroid'**, meaning a centroid with both pixels coordinates set to -1. This strategy allows the downstream nodes to identify target loss. Finally, the script includes the standard Python entry point structure to make it executable as a ROS 2 node.

The `main` function handles the node's lifecycle. It begins by initializing the ROS 2 client library using `rclpy.init()`. Then, it instantiates the `Detector` node object. The core of the function is the `try/except/finally` block: it enters the ROS 2 event loop, keeping the node alive to receive messages and execute callbacks until the node is shut down by the user, then it ensures graceful shutdown: upon interruption, the `node.destroy_node()` method is called to release ROS 2 resources, followed by `rclpy.shutdown()` to close the client library communication.

The `if __name__ == '__main__':` check is a standard Python idiom. It ensures that the `main()` function is called only when the script is executed directly.

5.2 Localizer

The *Localizer* node acts as the bridge between 2D image perception and the 3D world. Its primary role is to estimate the 3D position of the detected human target in the camera's coordinate frame by combining the pixel coordinates from the *Perceptor* with depth information from the robot's depth camera.

Logic pipeline

1. **Subscribes** to the centroid coordinates topic `/centroid_coords` published by *Perceptor*.
2. **Subscribes** to the depth point cloud topic `/camera/depth/points`.
3. Uses an **Approximate Time Synchronizer** to receive pairs of centroid and point cloud messages that are close in time.
4. In the callback, checks if the received centroid is valid or an 'error centroid'.
5. If the centroid is valid, it extracts the corresponding 3D point (x, y, z) from the point cloud using the pixel coordinates (u, v) to compute the corresponding index.
6. If the centroid is an 'error centroid', it generates a predefined 'error point'.
7. **Publishes** the resulting 3D point (either valid or error) as a `geometry_msgs/PointStamped` message on the `/target_position_camera` topic.

Code analysis

```
1 import rclpy
2 from rclpy.node import Node
3
4 from sensor_msgs.msg import PointCloud2
5 from geometry_msgs.msg import PointStamped
6 from doom_interfaces.msg import CentroidCoords
7
```

```
8 from message_filters import Subscriber, ApproximateTimeSynchronizer
9 import sensor_msgs_py.point_cloud2 as pc2
10
11 import numpy as np
12
13
14 class Localizer(Node):
15
16     def __init__(self):
17         super().__init__('localizer')
18
19         self.sub_centroid = Subscriber(self, CentroidCoords, '/centroid_coords
20         ↪ ')
21         self.sub_depth = Subscriber(self, PointCloud2, '/camera/depth/points')
22
23         # Initialize approximate time synchronizer to synchronize the two
24         ↪ different messages
25         ats = ApproximateTimeSynchronizer(
26             [self.sub_centroid, self.sub_depth],
27             queue_size=10,
28             slop=0.1
29         )
30         ats.registerCallback(self.localize)
31
32         self.publisher_ = self.create_publisher(
33             PointStamped,
34             '/target_position_camera',
35             10
36         )
37
38         self.get_logger().info(f"Localization node ready \n")
39
40     def localize(self, msg_centroid, msg_depth):
41
42         u = msg_centroid.u
43         v = msg_centroid.v
44
45         # If the 'error centroid' has been received, forward the 'error point
46         ↪ ', otherwise
47         # normally extract the depth point from the pointcloud
48         if u < 0 or v < 0:
49             x = 0.0
50             y = 0.0
51             z = -1.0
52         else:
53             height = msg_depth.height
54             width = msg_depth.width
55             # The pointcloud has linear indexing
56             index = v * width + u
57             points_array = pc2.read_points_numpy(msg_depth, field_names=("x",
58             ↪ "y", "z"))
59             x, y, z = points_array[index]
60
61         msg = PointStamped()
62         msg.point.x = float(x)
63         msg.point.y = float(y)
```

```
60     msg.point.z = float(z)
61     msg.header = msg_depth.header
62
63     self.publisher_.publish(msg)
64
65
66
67
68 def main(args=None):
69     rclpy.init(args=args)
70     node = Localizer()
71     try:
72         rclpy.spin(node)
73     except KeyboardInterrupt:
74         pass
75     finally:
76         node.destroy_node()
77         rclpy.shutdown()
78
79
80 if __name__ == '__main__':
81     main()
```

The class **constructor** initializes the **localizer** node. It sets up two **message_filters.Subscriber** instances, one for the **CentroidCoords** messages from the *Perceptor* and another for the **PointCloud2** messages from the depth sensor. Crucially, it initializes an **ApproximateTimeSynchronizer**. This synchronizer ensures that the **localize** callback function is only called when a pair of centroid and point cloud messages with closely matching timestamps (within the **slop** parameter value, here 0.1 seconds) are received. This synchronization is vital for accurately associating a 2D detection with its corresponding 3D depth data. Finally, the constructor creates a publisher for the resulting **PointStamped** message.

The **localize** callback is the core logic. It receives the synchronized centroid and point cloud messages. It first checks if the centroid coordinates **u** and **v** are negative, which signifies the 'error centroid' sent by the *Perceptor* when no person is detected: in this case, it constructs and publishes a **PointStamped** message with **z = -1.0** to propagate this error signal downstream. If the centroid is valid, it calculates the linear **index** corresponding to the pixel coordinates within the point cloud structure (using **v * width + u**). It then uses the **pc2.read_points_numpy** function to read the *entire* point cloud into a NumPy array and extracts the single 3D point (**x, y, z**) at the calculated **index**. Finally, it constructs a **PointStamped** message using the determined coordinates (either valid 3D point or error point), sets its header using the timestamp from the input **msg_depth** for consistency, and publishes it.

Finally, the script includes the usual Python entry point structure.

5.3 Navigator

The *Navigator* node is the central controller responsible for the robot's autonomous movement. It orchestrates the dynamic target following behavior by processing the 3D target position provided by the *Localizer*, determining the appropriate goal pose, and interacting with the ROS 2 Navigation Stack (Nav2) via its Action interface to command the robot.

Logic pipeline

1. **Subscribes** to the target position topic `/target_position.camera` published by *Localizer*.
2. Uses a **Timer** to periodically trigger the main logic.

3. The timer callback (navigate):

- Handles the 'error point' ($z = -1.0$), which indicates **target loss**, by canceling any active Nav2 goal and resetting the internal state.
- Combines the target position and the robot current position to **compute the next potential goal pose** and the relative straight-line distance.
- If the **next goal is too close**, then the robot will only rotate towards the goal, aiming at the target, but avoiding getting any closer, in order to keep a safety distance with the person.
- If a goal is already **in progress**:
 - If the computed next goal is significantly different from the current one, then cancels the current goal and sets the new one.
 - Otherwise, if the goals are too similar, does nothing, maintaining the current goal that Nav2 was already following.
- If no goal is in progress, simply sends the computed goal to Nav2.

Code analysis

```
1 import rclpy
2 from rclpy.node import Node
3 from rclpy.time import Time
4 from rclpy.duration import Duration
5 from rclpy.action import ActionClient
6
7 import math
8
9 from geometry_msgs.msg import PointStamped, PoseStamped, Quaternion
10 from tf2_geometry_msgs.tf2_geometry_msgs import do_transform_point
11
12 from tf2_ros.buffer import Buffer
13 from tf2_ros.transform_listener import TransformListener
14 from tf2_ros import TransformException
15
16 from nav2_msgs.action import NavigateToPose
17 from action_msgs.msg import GoalStatus
18
19
20 class Navigator(Node):
21
22     def __init__(self):
23         super().__init__('navigator')
24
25         self.tf_buffer = Buffer()
26         self.tf_listener = TransformListener(self.tf_buffer, self)
27
28         # Transform-related variables
29         self.camera_frame = 'camera_rgb_frame'
30         self.robot_frame = 'base_link'
31         self.global_frame = 'map'
32         self.transform_timeout = rclpy.duration.Duration(seconds=0.5)
33
34         # Null point definition, utils
35         self.null_point = PointStamped()
```

```
36     self.null_point.point.x = 0.0
37     self.null_point.point.y = 0.0
38     self.null_point.point.z = 0.0
39
40     # Node 'internal state'
41     self.person_position_camera = None
42     self.goal_in_progress = False
43     self.current_goal_handle = None
44     self.current_goal = None
45
46     # Safety distance from the target
47     self.stopping_distance = 0.75
48
49     # Update tolerances
50     self.distance_update_threshold = 0.05
51     self.yaw_update_threshold = 0.05
52
53     self.subscriber_ = self.create_subscription(
54         PointStamped,
55         '/target_position_camera',
56         self.save_point,
57         10
58     )
59
60     self._action_client = ActionClient(self, NavigateToPose, '
61         ↪ navigate_to_pose')
62
63     self.timer = self.create_timer(1.0, self.navigate)
64
65     self.get_logger().info(f"Navigator node ready \n")
66
67     def save_point(self, msg):
68
69         self.person_position_camera = msg
70
71     def navigate(self):
72
73         # Check if the target point has already been published
74         if self.person_position_camera is None:
75             return
76
77         # Check if the 'error point' has been received, in this case stop the
78         ↪ robot
79         if self.person_position_camera.point.z == -1.0:
80             self.get_logger().warn("Target lost. Stopping robot.")
81             if self.goal_in_progress and self.current_goal_handle:
82                 self.current_goal_handle.cancel_goal_async()
83                 self.goal_in_progress = False
84                 self.current_goal_handle = None
85                 self.current_goal = None
86             return
87
88         # Transform the target point from the robot camera frame to the global
89         ↪ map frame
90         try:
91             t = self.tf_buffer.lookup_transform(
```

```
89         self.global_frame,
90         self.camera_frame,
91         self.person_position_camera.header.stamp,
92         timeout=self.transform_timeout
93     )
94     person_position = do_transform_point(self.person_position_camera,
95         ↪ t)
96 except TransformException as ex:
97     return
98
99 # Retrieve robot current position wrt the global map frame
100 robot_position = self.get_robot_position(self.global_frame)
101 if robot_position is None:
102     self.get_logger().warn("Could not get robot position, skipping
103         ↪ navigation.")
104     return
105
106 # Compute the new goal, goal SLD distance
107 self.next_goal, goal_distance = self.compute_goal(robot_position,
108     ↪ person_position)
109
110 # If the SLD to the goal is less than a threshold, just rotate towards
111     ↪ the goal
112 if goal_distance < self.stopping_distance:
113     self.next_goal.pose.position = robot_position.point
114
115 # Reset the goal if one goal was already up, otherwise simply set the
116     ↪ goal
117 if self.goal_in_progress:
118     # Skip the goal reset if the new goal is similar to the old one
119     if not self.are_goals_different(self.current_goal, self.next_goal)
120         ↪ :
121         return
122     # Cancel the previous goal
123     cancel_future = self.current_goal_handle.cancel_goal_async()
124     # Reset the goal as soon as it's cancelled using the callback
125     cancel_future.add_done_callback(self.goal_canceled_callback)
126 else:
127     self.send_goal(self.next_goal)
128
129 def are_goals_different(self, old_goal, new_goal):
130
131     # If one of the two goals does not exist, then the new goal of course
132     ↪ is different
133     if old_goal is None or new_goal is None:
134         return True
135
136     # Check linear distance between goals
137     dist = math.hypot(
138         new_goal.pose.position.x - old_goal.pose.position.x,
139         new_goal.pose.position.y - old_goal.pose.position.y
140     )
141     if dist > self.distance_update_threshold:
142         return True
143
144     # Check angular distance between goals
```

```
138     old_yaw = self.quaternion_to_yaw(old_goal.pose.orientation)
139     new_yaw = self.quaternion_to_yaw(new_goal.pose.orientation)
140     yaw_diff = abs(self.normalize_angle(old_yaw - new_yaw))
141     if yaw_diff > self.yaw_update_threshold:
142         return True
143
144     # If none of the previous conditions is satisfied, then the new goal
145     # ↪ is very close to the old one
146     return False
147
148 def get_robot_position(self, source_frame):
149     try:
150         t = self.tf_buffer.lookup_transform(
151             source_frame,
152             self.robot_frame,
153             Time(),
154             self.transform_timeout
155         )
156         # The transform uses the null point since the robot in its own
157         # ↪ frame is located of course in
158         # the null coordinates
159         self.null_point.header.frame_id = self.robot_frame
160         self.null_point.header.stamp = Time().to_msg()
161         robot_position = do_transform_point(self.null_point, t)
162         return robot_position
163     except TransformException as ex:
164         return
165
166 def compute_goal(self, robot_position, target_position):
167     robot_x = robot_position.point.x
168     robot_y = robot_position.point.y
169     target_x = target_position.point.x
170     target_y = target_position.point.y
171     dx = target_x - robot_x
172     dy = target_y - robot_y
173
174     goal_distance = math.hypot(dx, dy)
175
176     yaw = math.atan2(dy, dx)
177     q = self.yaw_to_quaternion(yaw)
178
179     goal_pose = PoseStamped()
180     goal_pose.pose.position.x = target_x
181     goal_pose.pose.position.y = target_y
182     goal_pose.pose.position.z = 0.0
183     goal_pose.pose.orientation = q
184     goal_pose.header.frame_id = self.global_frame
185     goal_pose.header.stamp = self.get_clock().now().to_msg()
186
187     return goal_pose, goal_distance
188
189 def goal_canceled_callback(self, future):
190
```

```
191     # If the goal has been successfully canceled, send the new_goal and
192     ↪ reset the state,
193     # otherwise simply reset the state
194     try:
195         cancel_response = future.result()
196         self.goal_in_progress = False
197         self.current_goal_handle = None
198         self.current_goal = None
199         self.send_goal(self.next_goal)
200     except:
201         self.goal_in_progress = False
202         self.current_goal_handle = None
203         self.current_goal = None
204
205     def send_goal(self, goal_pose):
206
207         goal_msg = NavigateToPose.Goal()
208         goal_msg.pose = goal_pose
209
210         self.get_logger().info(f'Sending goal: {goal_pose.pose.position.x}, {
211         ↪ goal_pose.pose.position.y}')
212
213         self._action_client.wait_for_server()
214
215         # Asynchronously send goal
216         self._send_goal_future = self._action_client.send_goal_async(goal_msg)
217
218         # Add callback to be executed when the goal request has been served
219         self._send_goal_future.add_done_callback(self.goal_response_callback)
220
221     def goal_response_callback(self, future):
222
223         goal_handle = future.result()
224
225         # If the goal has not been accepted, log and reset the internal state
226         if not goal_handle.accepted:
227             self.get_logger().warn('Goal rejected')
228             self.goal_in_progress = False
229             self.current_goal_handle = None
230             self.current_goal = None
231             return
232
233         # If the goal has not been accepted, log and set the internal state
234         self.get_logger().info('Goal accepted')
235         self.goal_in_progress = True
236         self.current_goal_handle = goal_handle
237         self.current_goal = self.next_goal
238
239         # Asynchronously ask for navigation results
240         self._get_result_future = goal_handle.get_result_async()
241
242         # Add callback to be executed when the results request has been served
243         self._get_result_future.add_done_callback(self.get_result_callback)
244
245     def get_result_callback(self, future):
```



```
245
246     self.goal_in_progress = False
247     self.current_goal_handle = None
248     self.current_goal = None
249
250     @staticmethod
251     def yaw_to_quaternion(yaw):
252
253         q = Quaternion()
254         q.w = math.cos(yaw / 2.0)
255         q.z = math.sin(yaw / 2.0)
256         q.x = 0.0
257         q.y = 0.0
258         return q
259
260     @staticmethod
261     def quaternion_to_yaw(q):
262
263         yaw = 2.0 * math.atan2(q.z, q.w)
264         return yaw
265
266     @staticmethod
267     def normalize_angle(angle):
268
269         while angle > math.pi:
270             angle -= 2.0 * math.pi
271         while angle < -math.pi:
272             angle += 2.0 * math.pi
273         return angle
274
275
276
277
278 def main(args=None):
279     rclpy.init(args=args)
280     node = Navigator()
281     try:
282         rclpy.spin(node)
283     except KeyboardInterrupt:
284         pass
285     finally:
286         node.destroy_node()
287         rclpy.shutdown()
288
289
290 if __name__ == '__main__':
291     main()
```

The class **constructor** initializes the **navigator** node. It sets up the necessary **tf2** buffer and listener for coordinate transformations, defining key frame names (**camera_frame**, **robot_frame**, **global_frame**) and a transformation timeout. It initializes state variables: **person_position_camera** to store the latest target position from the subscriber, **goal_in_progress**, a boolean flag to track if a goal is currently being executed, **current_goal_handle** to store the action server goal handle and **current_goal** to store the actual **PoseStamped** message corresponding to the goal Nav2 is currently pursuing. Thresholds are defined: **stopping_distance** determines the safety distance, while **distance_update_threshold** and **yaw_update_threshold** control the sensitivity for goal updates. A subscriber is created for the

/target_position_camera topic, using the simple `save_point` callback. An Action Client (`_action_client`) is initialized to communicate with the Nav2 `navigate_to_pose` action server. Finally, a **Timer** is created to periodically execute the main `navigate` method every 1.0 second.

The `save_point` callback simply saves the target position in the camera frame, received from the localizer node, in the node state.

The primary logic resides in the `navigate` method. Triggered by the timer, it first checks if a target point (`person_position_camera`) has been received. It then checks if the received point is an 'error point' ($z = -1.0$). If so, it logs a warning, cancels any active goal using the stored `current_goal_handle`, resets the internal state (`goal_in_progress`, `current_goal_handle`, `current_goal`), and returns. If the point is valid, it attempts to transform it from the camera frame to the global map frame using `lookup_transform` and `do_transform_point`. On failure, it returns. It then calls `get_robot_position` to find the robot's current pose in the map frame; if this fails, it returns. With both target and robot positions available, it calls `compute_goal`, storing the calculated goal pose in `self.next_goal` and getting the straight-line `goal_distance`. It checks if `goal_distance` is below `stopping_distance`; if true, it modifies the position component of `self.next_goal` to the robot's current position, effectively creating an in-place rotation goal, which allow the robot to always aim at the target, even if it's close to it. The crucial update logic follows: if `goal_in_progress` is `True`, it compares the newly computed `self.next_goal` with the currently active `self.current_goal` using `are_goals_different`. If they are too similar, it returns, avoiding a useless update. If they differ significantly, it requests cancellation via `current_goal_handle.cancel_goal_async` and uses a callback which calls `send_goal` with the new `self.next_goal` once the cancellation is complete. If no goal was initially in progress, it simply calls `send_goal` with the computed `self.next_goal`.

The `get_robot_position` method retrieves the robot's pose in the specified frame. It does this by getting the transform from `self.robot_frame` (base.link) to the `target_frame` (map) and applying it to the origin point (0,0,0). It returns the resulting `PointStamped` or `None` on failure.

The `compute_goal` method calculates the target pose and the straight-line distance. Given robot and target positions, it computes the distance using `math.hypot` and the required yaw using `math.atan2` in order to have a resulting pose where the robot will face the target. The yaw is then converted to a `Quaternion` via the static helper `yaw_to_quaternion` and these values are used to populate a `PoseStamped` message, which is returned along with the computed distance.

The `are_goals_different` helper method compares two goal poses (`old_goal` and `new_goal`, both `PoseStamped` messages) to determine if they are significantly different, based on predefined thresholds. First, it handles the edge case where either goal is `None`, returning `True` (indicating they are different). It then calculates the Euclidean distance between the (x, y) positions of the two goals using `math.hypot`. If this distance exceeds `self.distance_update_threshold`, the function immediately returns `True`. If the distance is within the threshold, it proceeds to compare the orientation. It converts the `Quaternion` orientation of both goals into yaw angles using the `quaternion_to_yaw` static helper. The difference between these yaw angles is calculated and then normalized to the range $[-\pi, \pi]$ using the `normalize_angle` static helper to find the shortest angular distance. The absolute value of this normalized difference is compared against `self.yaw_update_threshold`. If the yaw difference exceeds this threshold, the function returns `True`. If both the distance and the normalized yaw difference are within their respective thresholds, the function returns `False`, indicating the goals are considered too similar to warrant an update.

Action client interactions are managed via asynchronous calls and callbacks. `send_goal` packages the `PoseStamped` into a `NavigateToPose.Goal` and sends it using `_action_client.send_goal_async`. It registers `goal_response_callback` to handle the server's acceptance or rejection.

The `goal_response_callback` is triggered when the server responds to the `send_goal_async` request. It checks `goal_handle.accepted`. If rejected, it logs a warning and resets the node's state (`goal_in_progress`, `current_goal_handle`, `current_goal`). If accepted, it logs confirmation, sets `goal_in_progress` to `True`, stores the received `goal_handle` in `current_goal_handle`, and, crucially, updates `self.current_goal` to match the `self.next_goal` that was just sent and accepted. It then calls `goal_handle.get_result_async` and registers `get_result_callback` to handle the final result.

The `get_result_callback` is invoked when the navigation action concludes. It resets the node's state variables (`goal_in_progress`, `current_goal_handle`, `current_goal`), indicating the action is no longer active.

Static helper methods `yaw_to_quaternion`, `quaternion_to_yaw`, and `normalize_angle` provide essential angle conversion and normalization utilities, used in `compute_goal` and `are_goals_different`.

Again, the script includes the usual Python entry point structure.

5.4 Initial position publisher

6 Launch File